

Luc SARRAZIN

BTS SIO 2^{ème} année



Rapport de stage

Janin Consulting

Développeur Full-Stack :

Démarche pour la création un bot IA capable
de simuler des entretiens interactifs

20/01/2025 au 14/03/2025

Sommaire

Introduction.....	3
Développement.....	4
1. Développement Web (Front-End)	4
1.1 Conception du bot IA	4
1.2 Programmation des pages (Angular et HTML/CSS)	6
1.3 Débogage du Front-End	9
2. Développement Java (Back-End)	10
2.1 Créations des Services et Controllers de l'API	10
2.2 Débogage de l'API.....	12
3. Répartition du Travail	13
3.1 Tâches Assignées	13
Conclusion.....	14
Annexe	15

Introduction

Dans le cadre de ma formation en BTS Services Informatiques aux Organisations, spécialité Solutions Logicielles et Applications Métiers (SLAM), j'ai effectué un stage au sein de l'entreprise Janin Consulting afin de continuer le projet Ask, du 20 février au 14 mars 2025.

Les objectifs de ce stage étaient d'acquérir une expérience professionnelle afin de mettre en pratique et de consolider les connaissances obtenues tout au long de ma formation en BTS SIO. Grâce à cette expérience, j'ai pu approfondir mes compétences en développement web et en intelligence artificielle appliquée.

Ce stage m'a permis de travailler sur un projet concret en développant un bot IA capable de simuler des entretiens interactifs basés sur le CV du candidat et les attentes de l'appel d'offre. J'ai été responsable de l'ensemble du projet, ce qui m'a amené à concevoir et développer à la fois le front-end en Angular et le back-end en Java Spring Boot.

L'application Ask est une solution SaaS qui optimise l'interaction entre recruteurs et candidats grâce à l'intelligence artificielle. L'objectif du bot que j'ai développé était de fournir une expérience immersive et personnalisée aux candidats, en leur permettant de s'entraîner aux entretiens grâce à un système de questions adaptatives, de feedback en temps réel et d'analyse des réponses.

L'une des principales missions qui m'a été confiée était de concevoir une interface ergonomique pour la simulation d'entretien, ainsi qu'un système backend capable de gérer la génération des questions, l'analyse des réponses et la production d'un rapport PDF final pour le candidat.

Développement

1. Développement Web (Front-End)

1.1 Conception du bot IA

Le bot 2 est conçu pour offrir une simulation d'entretien interactif basée sur l'appel d'offre et le CV validé par le bot 1. Il vise à préparer efficacement le candidat en lui posant des questions adaptées, réparties en trois catégories :

Techniques : évaluation des compétences métiers.

Comportementales : analyse du savoir-être professionnel.

Expériences spécifiques : approfondissement des missions réalisées.

Le commercial pourra configurer différents types de questions pour le bot :

Choix multiple (checkbox) : plusieurs réponses possibles.

Choix unique (radio button) : une seule réponse correcte.

Questions ouvertes (textarea) : réponse libre du candidat.

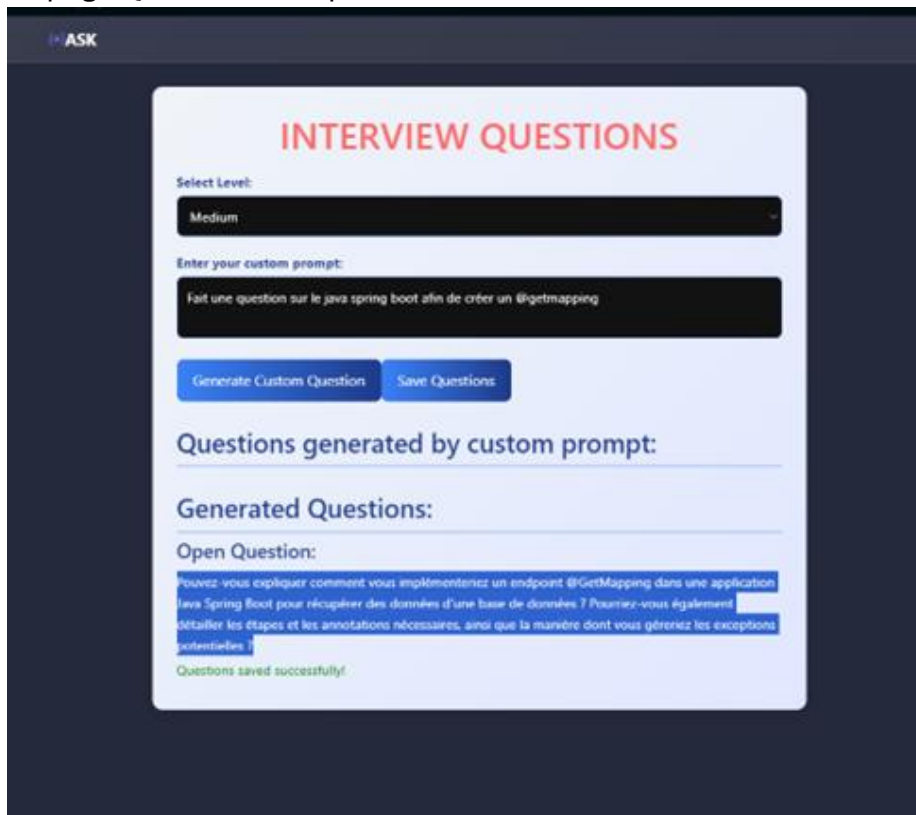
Le bot ajustera les questions en fonction des réponses et proposera des aides en cas de blocage. Chaque réponse sera analysée pour aider le candidat à structurer son discours, avec un score et un rapport PDF final contenant des recommandations.

Voici une maquette de la page :



À la suite de notre maquette initiale, j'ai dû créer deux pages essentielles pour le bon fonctionnement du bot : Question Prompt et Job Interview.

La page Question Prompt :



INTERVIEW QUESTIONS

Select Level:
Medium

Enter your custom prompt:
fait une question sur le java spring boot afin de créer un @GetMapping

Generate Custom Question Save Questions

Questions generated by custom prompt:

Generated Questions:

Open Question:

Pouvez-vous expliquer comment vous implémenteriez un endpoint @GetMapping dans une application Java Spring Boot pour récupérer des données d'une base de données ? Pouvez-vous également détailler les étapes et les annotations nécessaires, ainsi que la manière dont vous gérez les exceptions potentielles ?

Questions saved successfully!

La page Job Interview :



Simulation d'Entretien

En considérant votre expérience en tant que développeur Full-Stack chez Janin Consulting, où vous avez travaillé sur des projets utilisant Angular, Spring Boot et MySQL, pouvez-vous nous expliquer comment vous avez abordé l'intégration d'une API avec une solution d'intelligence artificielle dans votre projet de création d'un site web et d'une API pour un système connecté aux Ressources Humaines? Quels défis avez-vous rencontrés et comment les avez-vous surmontés?

Vous:

Avez-vous déjà travaillé sur un projet où vous deviez intégrer une API avec une solution d'intelligence artificielle, similaire à ce qui est mentionné dans votre expérience chez Janin Consulting ? Si oui, pouvez-vous expliquer comment vous avez utilisé Angular et Spring Boot dans ce contexte ?

Avez-vous déjà travaillé sur un projet où vous deviez intégrer une API avec une solution d'intelligence artificielle, similaire à ce qui est mentionné dans votre expérience chez Janin Consulting ? Si oui, pouvez-vous expliquer comment vous avez utilisé Angular et Spring Boot dans ce contexte ?

Oui
Non

Envoyer

1.2 Programmation des pages (Angular et HTML/CSS)

Pour créer les différentes pages, j'ai utilisé Visual Studio Code, ce qui a permis de manipuler le HTML, le CSS et Angular (qui utilise TypeScript). Sur l'image ci-dessus, on retrouve la fonction qui permet d'évaluer les réponses de l'utilisateur sur une question répondus.

```
391 evaluateResponses(): Promise<void> {
392   return new Promise<(resolve) => {
393     const evaluationPromises: Promise<void>[] = this.Reponses.map((reponse) => {
394       return new Promise<void>((innerResolve) => {
395         this.apiService.evaluateResponse(reponse.question, reponse.reponse, reponse.type, reponse.choice, reponse.cv, reponse.offre).subscribe({
396           next: (evaluation) => {
397             console.log('Évaluation de la réponse...');
398             console.log(evaluation);
399
400             this.questions.push(evaluation);
401             this.answers.push({
402               question: reponse.question,
403               response: reponse.reponse,
404               score: evaluation.pass ? 1 : 0,
405               feedback: evaluation.explanation,
406               choice: reponse.choice
407             });
408
409             if (!evaluation.pass) {
410               this.incorrectQuestions.push(reponse);
411             }
412
413             innerResolve();
414           },
415           error: (err) => {
416             console.error('Erreur lors de l\'évaluation de la réponse:', err);
417             innerResolve();
418           }
419         });
420       });
421     });
422   }
```

J'ai utilisé un framework, Bootstrap, qui a facilité la création des pages et de l'esthétique, notamment avec les icônes. On retrouve également une implémentation simplifiée pour faire afficher sur le html les résultats.

```
<!-- Écran des questions -->
<div *ngIf="isInterviewStarted && !isInterviewEnded" id="question-screen" class="text-center">
  <div id="conversation" class="mb-3">
    <div *ngFor="let msg of conversation" [ngClass]="{'bot-message': msg.sender === 'bot', 'candidate-message': msg.sender === 'user'}">
      <div *ngIf="msg.sender === 'bot'" class="bot-message">
        
        <strong>Bot: </strong> {{ msg.question }}{{ msg.text }}
      </div>
      <div *ngIf="msg.sender === 'user'" class="candidate-message">
        <strong>Vous: </strong> {{ msg.text }}
        
      </div>
    </div>
  </div>
  <div *ngIf="currentQuestion">
    <h3>{{ currentQuestion.text }}</h3>
  </div>
</div>
```

L'implémentation a été compliquée et j'ai dû chercher de l'aide dans les documentations, car l'imbrication des NgIf et des NgFor a été compliquée, car ces directives ne captaient pas correctement certaines données provenant du back-end.

Ceci est la fonction dans l'api service afin d'envoyer nos données dans le back afin de les recevoir et pouvoir renvoyer la réponse d'Open AI :

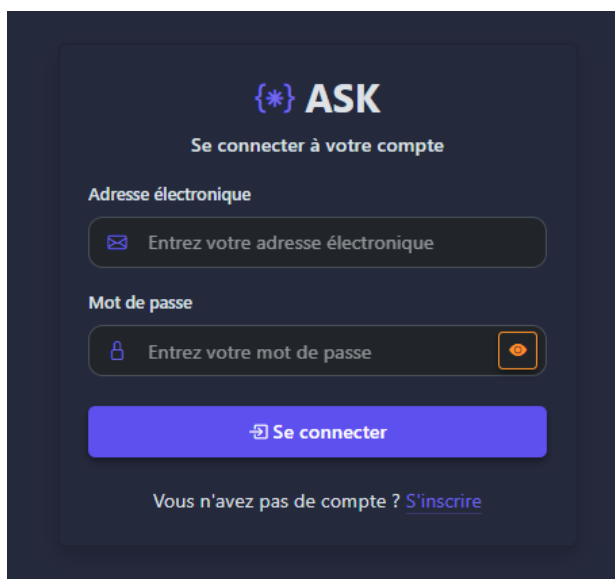
```
public evaluateResponse(question: string, response: string, type: string, choice : string, resumeData: any, callOfTenders:any): Observable<any> {
  console.log('resumeData', resumeData, 'callOfTenders', callOfTenders);
  const params = new HttpParams()
    .set('question', question)
    .set('response', response)
    .set('type', type.toLowerCase())
    .set('cv', resumeData)
    .set('offre', callOfTenders)
    .set('choice', choice);
  const body = {
    resumeData,
    callOfTenders
  }
  return this.httpClient.post<any>(`${this.baseUrl}/evaluate`, body, { params },);
}
```

On retrouve donc dans cette méthode tous les paramètres nécessaires à l'évaluation d'une réponse. Une fois cette fonction implémentée, elle nous a servi de base pour les autres appels API. Ainsi, nous avons pu réutiliser la logique de construction des paramètres et l'envoi des requêtes HTTP pour d'autres fonctionnalités similaires.

Nous avons également mis en place un design responsive en utilisant du CSS grâce aux classes et ID, ce qui permet d'ajuster les éléments de la page pour le format téléphone. Les cours sur le CSS durant l'année nous ont permis d'acquérir des connaissances sur le responsive, ce qui nous a permis de les mettre en œuvre durant le stage.

Pour la page Modèle, nous avons prévu avec le tuteur et le client de rajouter un système de géolocalisation qui permettrait de proposer aux modèles des offres proches de leur localisation. Cependant, nous n'avons pas pu continuer cette fonctionnalité car elle n'était finalement pas prioritaire. Nous avons quand même pu aborder cette option avec Angular/TypeScript en ajoutant un code qui récupère la localisation de l'utilisateur en demandant l'accès via une popup.

Voici le système de d'authentification :



The image shows a login form titled "ASK" with a logo consisting of a curly brace and an asterisk. Below the title is the text "Se connecter à votre compte". The form has two input fields: "Adresse électronique" with an email icon and "Mot de passe" with a lock icon and a toggle eye icon. Below these fields is a blue button labeled "Se connecter". At the bottom, there is a link that says "Vous n'avez pas de compte ? S'inscrire".

Ceci est le système qui nous permet de nous connecter à nos comptes et d'accéder à nos pages, qui sont accessibles uniquement selon nos rôles. Par exemple, avec ce compte qui est un compte recruteur, j'aurais accès aux pages RECRUITER.

Nous avons mis en place tout un système d'authentification et de redirection de pages intégré dans notre programme.

```
const routes: Routes = [
  { path: '', component: HomePageComponent },
  { path: 'sign-in', component: SignInPageComponent },
  { path: 'sign-up', component: SignUpPageComponent },
  { path: 'interview', component: JobInterviewComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'PromptQuestion', component: QuestionPromptComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'interview2', component: InterviewComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },

  { path: 'resumes', component: ResumesPageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'call-of-tenders', component: CallOfTendersPageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'reports', component: ReportsPageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'resume-template', component: ResumeTemplatePageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'profile', component: ProfilePageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },
  { path: 'bot', component: BotPageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE', 'RECRUITER', 'ADMIN'] } },

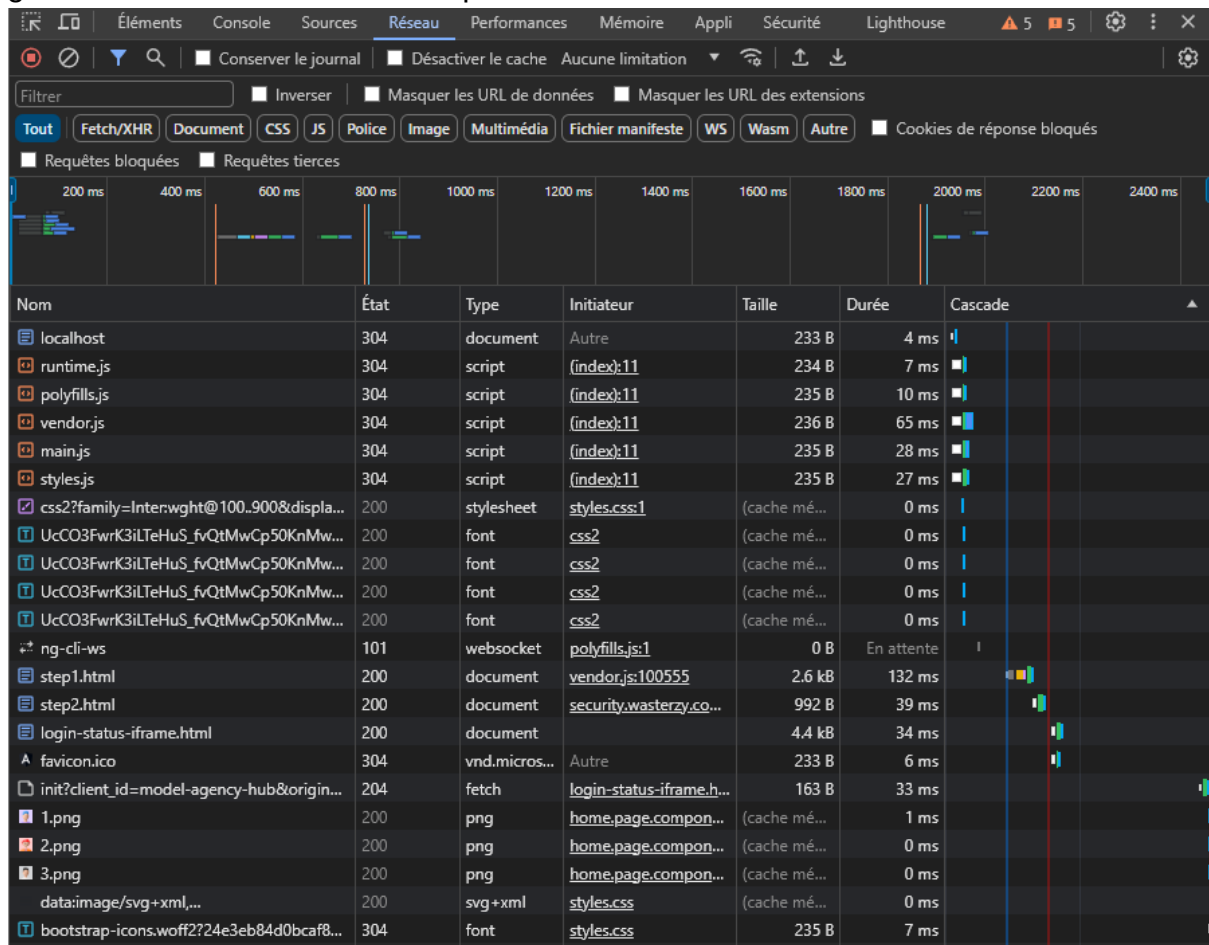
  { path: 'vs-card', component: VsCardCandidatePageComponent, canActivate: [AuthGuard], data: { roles: ['CANDIDATE'] } },

  // otherwise redirect to landing
  { path: '**', redirectTo: '' }
];
```

Ainsi, pour chaque nouvelle page créée, il faut définir une route en la reliant à AuthGuard. Cela nous permet, lorsqu'on se connecte, d'atterrir sur la bonne page correspondant à notre rôle.

1.3 Débogage du Front-End

Le stage m'a appris à inspecter une page dans mon navigateur internet, ce qui m'a grandement aidé à identifier les problèmes et erreurs dans le code.



Nom	État	Type	Initiateur	Taille	Durée	Cascade
localhost	304	document	Autre	233 B	4 ms	
runtime.js	304	script	(index):11	234 B	7 ms	
polyfills.js	304	script	(index):11	235 B	10 ms	
vendor.js	304	script	(index):11	236 B	65 ms	
main.js	304	script	(index):11	235 B	28 ms	
styles.js	304	script	(index):11	235 B	27 ms	
css2?family=Inter:wght@100..900&displa...	200	stylesheet	styles.css:1	(cache mé...)	0 ms	
UcCO3FwrK3iLTeHuS_fvQtMwCp50KnMw...	200	font	css2	(cache mé...)	0 ms	
UcCO3FwrK3iLTeHuS_fvQtMwCp50KnMw...	200	font	css2	(cache mé...)	0 ms	
UcCO3FwrK3iLTeHuS_fvQtMwCp50KnMw...	200	font	css2	(cache mé...)	0 ms	
UcCO3FwrK3iLTeHuS_fvQtMwCp50KnMw...	200	font	css2	(cache mé...)	0 ms	
ng-cli-ws	101	websocket	polyfills.js:1	0 B	En attente	
step1.html	200	document	vendor.js:100555	2.6 kB	132 ms	
step2.html	200	document	security.wasterzy.co...	992 B	39 ms	
login-status-iframe.html	200	document		4.4 kB	34 ms	
favicon.ico	304	vnd.micros...	Autre	233 B	6 ms	
init?client_id=model-agency-hub&origin...	204	fetch	login-status-iframe.h...	163 B	33 ms	
1.png	200	png	home.page.compon...	(cache mé...)	1 ms	
2.png	200	png	home.page.compon...	(cache mé...)	0 ms	
3.png	200	png	home.page.compon...	(cache mé...)	0 ms	
data:image/svg+xml,...	200	svg+xml	styles.css	(cache mé...)	0 ms	
bootstrap-icons.woff2?24e3eb84d0bcaf8...	304	font	styles.css	235 B	7 ms	

En allant dans l'onglet Réseau, j'ai accès à toutes les informations entrantes et sortantes du site internet, telles que les logos Bootstrap ou les requêtes de recherche d'utilisateurs.

J'ai également utilisé cet outil pour récupérer le token d'authentification, nécessaire pour obtenir l'identifiant de l'utilisateur connecté. Chaque utilisateur génère automatiquement un token sécurisé de manière aléatoire.

Dans l'onglet Console, je peux voir toutes les erreurs ou avertissements de programmation grâce à des fonctions comme `console.log` ou `console.warning`. Cela fonctionne un peu comme un `System.out.println` en Java, utilisé pour le débogage ou pour afficher des messages informatifs pour l'utilisateur.

Pour résoudre les différentes erreurs rencontrées, cette interface m'a permis de copier les messages d'erreur afin de faire des recherches sur internet ou de demander de l'aide à mon tuteur pour trouver des solutions.

J'ai aussi nettoyé le front pour une meilleure visibilité des erreurs.

2. Développement Java (Back-End)

2.1 Créations des Services et Contrôlers de l'API

Durant mon stage, j'ai dû créer une API pour gérer diverses fonctionnalités, telles que la génération de questions par l'IA (ouvertes, fermées et à choix multiples), l'évaluation des réponses, la sauvegarde des questions et la génération de questions en fonction de l'offre.

Pour cela, mon tuteur m'a fourni un code GitHub contenant le début d'une API basée sur Java Spring Boot. J'ai ensuite complété cette base en développant des contrôleurs (controllers) et des services afin d'intégrer pleinement ces fonctionnalités.

Exemple de controller :

```
@PostMapping("/question/open")
public OpenQuestionDto generateOpenQuestion(@RequestParam String theme, Level level) {
    OpenQuestion question = generator.generateOpenQuestion(theme, level);
    return questionMapper.toDto(question);
}
```

Exemple de service :

```
@Override
public OpenQuestion generateOpenQuestion(String theme, Level level) {
    // Vérification des paramètres
    if (theme == null || level == null) {
        throw new IllegalArgumentException("Le thème et le niveau ne peuvent pas être null");
    }

    String systemPrompt = """
    Merci de générer une question ouverte d'entretien en suivant les directives ci-dessous.
    La question doit être pertinente par rapport au thème et au niveau spécifiés.
    Assurez-vous que la question soit claire et engageante pour le candidat sans dépasser un certain nombre de tokens.

    Règles :
    - Ne pas dépasser 500 tokens dans votre réponse.
    - Pour toutes les questions tu dois t'en tenir au CV et à l'offre pour l'évaluer et si il l'a bien répondu ou pas et posé une question pour lui faire réussir la quest
    - Merci de ne pas répondre à ces phrase d'introduction juste à la réponse du candidat par rapport à la question.
    - Merci de répondre seulement par rapport au CV et à l'offre si ça parle d'autre chose tu arrête la conversation en disant que ça sort du concept.
    """;

    String userPrompt = String.format("Thème : %s\nNiveau : %s", theme, level);

    Map<String, Object> params = Map.of(
        k1: "model", v1: "gpt-4o-2024-08-06", // Utiliser gpt-4 standard
        "messages", List.of(
            Map.of(k1: "role", v1: "system", k2: "content", systemPrompt),
            Map.of(k1: "role", v1: "user", k2: "content", userPrompt)
        ),
        k3: "temperature", v3: 0.7,
        k4: "max_tokens", v4: 500
    );

    // Créer un WebClient pour ajouter les headers nécessaires
    WebClient webClient = WebClient.builder()
        .baseUrl("https://api.openai.com/v1/chat/completions")
        .defaultHeader(HttpHeaders.CONTENT_TYPE, "application/json")
        .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + apiKey)
        .build();

    // Effectuer l'appel avec WebClient et récupérer la réponse
    Mono<String> responseMono = webClient.post().RequestBodyUriSpec
```

Ce code ci-dessus permet de générer une question ouverte d'entretien en envoyant une requête à l'API OpenAI. L'objectif est de récupérer une question pertinente

basée sur un thème, un niveau, ainsi que les informations du CV et de l'offre d'emploi. La question générée est ensuite renvoyée au format JSON pour être utilisée sur le front-end.

Vérification des paramètres : On s'assure que le thème et le niveau sont bien renseignés.

Création des instructions : On définit les règles que l'IA doit suivre pour générer une question adaptée au CV et à l'offre d'emploi.

Par exemple :

- La question doit être claire et en rapport avec le CV et l'offre d'emploi.
- Ne pas dépasser 500 tokens (pour gérer la taille et le temps de réaction de l'IA).

Envoi de la requête : On utilise WebClient pour interagir avec OpenAI et récupérer une réponse.

Traitement de la réponse : On extrait la question générée et on la convertit en objet OpenQuestion.

Affichage : La question est renvoyée au format JSON pour être affichée sur le front-end.

Ce processus automatise la création de questions d'entretien personnalisées en fonction du candidat.

Voici l'un des résultats qu'on reçoit dans la console :

```
2025-03-10T21:44:46.007+01:00 INFO 25792 --- [AskApiApplication] [ent-11-Worker-0] c.a.a.business.service.InterviewService : Appel API terminé
2025-03-10T21:44:46.007+01:00 INFO 25792 --- [AskApiApplication] [nio-8080-exec-9] c.a.a.business.service.InterviewService : {
  "id": "chatcmpl-B9eHLaStfRW7knjmlvraidsSRXEba",
  "object": "chat.completion",
  "created": 1741639483,
  "model": "gpt-4o-2024-08-06",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Pouvez-vous expliquer comment vous implémenteriez un endpoint @GetMapping dans une application Java Spring Boot pour récupérer des données d'une base de données ? Pourriez-vous également détailler le",
        "refusal": null
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 228,
    "completion_tokens": 58,
    "total_tokens": 286,
    "prompt_tokens_details": {
      "cached_tokens": 0,
      "audio_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 0,
      "audio_tokens": 0,
      "accepted_prediction_tokens": 0,
      "rejected_prediction_tokens": 0
    }
  },
  "service_tier": "default",
  "system_fingerprint": "fp_eb9dce56a8"
}
2025-03-10T21:44:46.007+01:00 INFO 25792 --- [AskApiApplication] [nio-8080-exec-9] c.a.a.business.service.InterviewService : Generated question text: Pouvez-vous expliquer comment vous implémenteriez un endpoint @GetM
2025-03-10T21:44:46.007+01:00 INFO 25792 --- [AskApiApplication] [nio-8080-exec-9] c.a.a.business.service.InterviewService : Generated OpenQuestion : OpenQuestion(response=null indications=null tokens=OPEN)
```

Le contrôleur est l'un des codes qui nous permettra de communiquer entre le Front et le Back. Le service est la ligne de code qui sera exécutée lorsque le contrôleur le sera. Par exemple, lorsque dans le contrôleur, `generateOpenQuestion` sera exécuté par le front, le contrôleur va exécuter la ligne dans le service `generateOpenQuestion` et va la transmettre au contrôleur, qui la transmettra au front.

2.2 Débogage de l'API

Notre tuteur nous a enseigné l'art du débogage avec IntelliJ IDEA, qui dispose d'une fonctionnalité intéressante : les breakpoints. Ces derniers permettent de suspendre l'exécution du programme à des points spécifiques, afin de pouvoir avancer pas à pas dans le code.

La console affiche les erreurs ainsi que les succès, par exemple lors de la recherche d'un utilisateur.

```
Hibernate: select ce1_0.id,ce1_0.creation_date,ce1_0.description,ce1_0.logo_base_64,ce1_0.name,ce1_0.updating_date from user_company uce1_0 join company ce1_0 on ce1_0.id=uce1_0.company_id where uce1_0.user_id=?
Hibernate: select ue1_0.id,ue1_0.creation_date,ue1_0.email,ue1_0.email_verified,ue1_0.first_name,ue1_0.last_name,ue1_0.password,ue1_0.updating_date from 'user' ue1_0 where ue1_0.email=?
Hibernate: select r1_0.user_id,r1_0.id,r1_0.creation_date,r1_0.role,r1_0.updating_date from user_role r1_0 where r1_0.user_id=?
```

3. Répartition du Travail

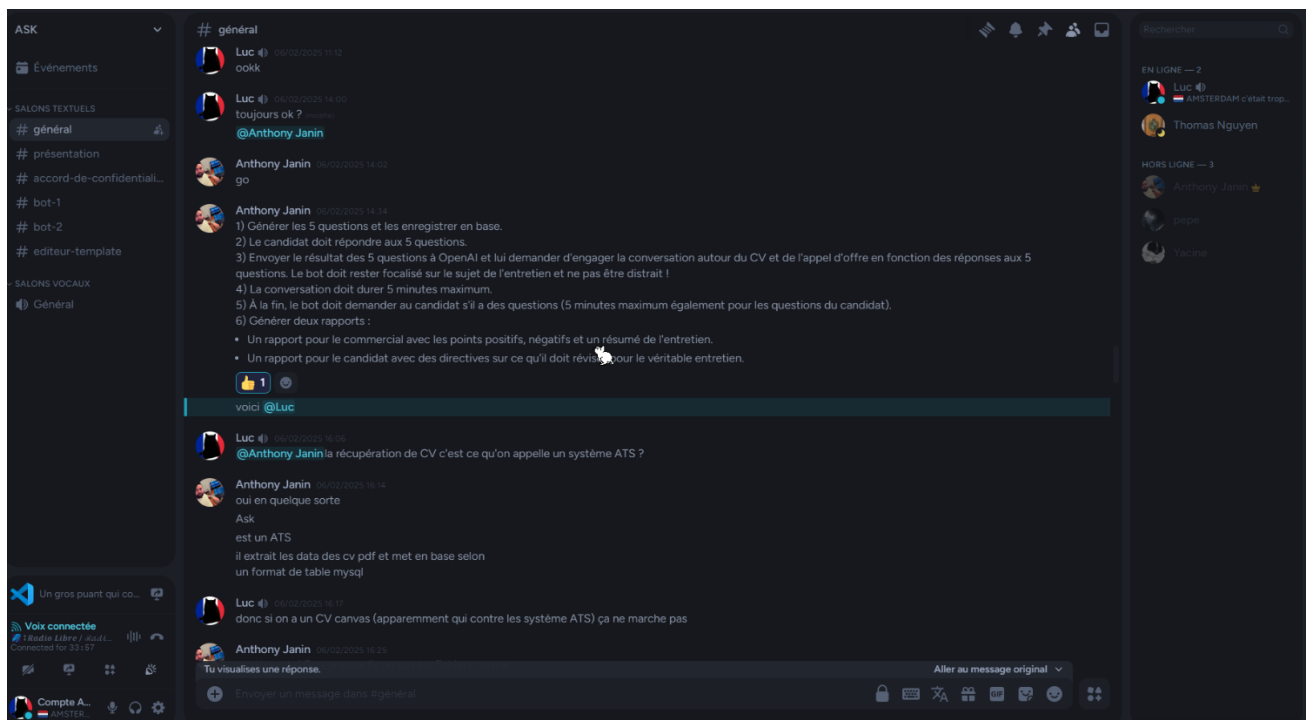
3.1 Tâches Assignées

Pendant mon stage, j'ai travaillé seul, ce qui m'a amené à gérer toutes les tâches de développement de manière autonome. Pour m'organiser, j'ai utilisé Visual Studio Code et IntelliJ IDEA afin de travailler efficacement sur les fichiers et modifier le code en fonction des besoins.

J'ai dû me concentrer sur plusieurs aspects du projet, notamment le développement de certaines pages et la récupération d'informations essentielles, comme l'ID de l'offre d'emploi et du CV. Travailler seul m'a permis de renforcer ma capacité à résoudre les problèmes de manière indépendante et à structurer mon travail de façon optimale.

Pour faciliter la communication avec mon tuteur et obtenir de l'aide en cas de besoin, j'ai utilisé l'application Discord. Un serveur dédié, mis à disposition par le tuteur, m'a permis d'accéder aux ressources du projet, notamment une présentation du réseau social et de ses différentes fonctionnalités. De plus, le tuteur m'a fourni quelques informations pour m'aider à mieux produire le produit final.

Voici une image du serveur discord :



Pendant mon stage, j'ai développé deux pages et j'ai également dû modifier certaines pages afin de récupérer des informations, comme l'ID de l'offre d'emploi et du CV.

Le seul inconvénient était que les fichiers étaient stockés sur mon ordinateur, ce qui signifiait qu'il n'avait pas accès à eux sans moi. Par exemple, la base de données SQL était différente, donc nous devons constamment synchroniser nos fichiers. Cela nécessitait une rigueur dans le partage des fichiers, heureusement, nous avons tous accès à un GitHub partagé par notre tuteur.

Conclusion

Ce stage de deux mois m'a offert une immersion précieuse dans le monde professionnel, notamment dans les aspects variés d'un projet. Travailler dans un environnement discipliné a renforcé ma maturité et ma responsabilité, des traits essentiels pour évoluer dans le domaine du développement de jeux vidéo.

Pendant cette période, j'ai exploré divers langages de programmation tels que HTML, CSS, Angular, Java Spring Boot, MySQL, Bootstrap. J'ai eu l'opportunité de mettre en pratique les connaissances et compétences acquises au cours de mes études en programmation. Ce projet m'a également initié à la création de sites web interactifs et à la gestion de bases de données ainsi que l'utilisation de l'IA dans un projet professionnel, des compétences cruciales dans le développement de jeux.

D'autre part, j'ai acquis de nouvelles compétences techniques comme le débogage avancé, la gestion d'API, la manipulation des tokens, et l'utilisation efficace de GitHub pour le versionnage de code. Cette expérience m'a également appris à rechercher activement des solutions aux problèmes techniques, renforçant ainsi mon autonomie et ma capacité d'adaptation.

Travailler seul sur ce projet m'a permis de mieux structurer mon travail et de gérer l'ensemble du développement de manière indépendante. J'ai dû organiser mes tâches efficacement pour atteindre mes objectifs dans les délais impartis.

Enfin, cette expérience a consolidé mon ambition de devenir programmeur de jeux vidéo. Je suis reconnaissant envers mon tuteur Anthony Janin pour son soutien et ses conseils précieux tout au long de ce stage enrichissant.

Annexe

Logiciels utilisés

- Visual studio Code
- IntelliJ Idea
- XAMPP

Langage de programmation utilisés

- HTML/CSS
- Java Spring BOOT
- Angular (Typescript)
- MySQL

Framework utilisés

- Bootstrap

Compétences Acquises

- Autonomie
- Recherche de solutions
- Organisation
- Travail d'équipe
- Adaptabilité

Liens utiles (Documentation) :

- Bootstrap Docs : <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- Bootstrap Icon : <https://icons.getbootstrap.com/>
- UUID in Java Spring Boot: <https://www.baeldung.com/java-uuid>